float[cut-set ; value-set]

cut-set: set of cuts (e.g., $\{<0.1, <0.5, \leq 0.8\}$) or $\top$ (uncut)

value-set: set of concrete values (e.g., $\{0.1, 0.4, 0.5\}$) or $\top$ (any value)

Fig. 3. The type **float**[cut-set; value-set] captures two aspects of a floating-point expression. The cut set is determined either by (i) the constants the expression is compared against or (ii) the constant itself if the expression is a float constant. The value set records the concrete values an expression can take. The discretization is driven by the cut set: the n cuts define $n + 1$ intervals over the real line.

Value sets refine an expression's cut set by adding cuts induced by comparisons. For example, in the test $x \leq y$, the value set of y contributes boundary points that must also appear in the cut set of x. This dependency is non-local: constants introduced in one branch can constrain how a sampled value from another branch must be discretized.

Whenever two expressions are compared, the type system equates their cut sets so they are discretized over the same intervals. These cut sets then propagate backward through the control flow to all contributing subexpressions. Figure 18 illustrates this non-local propagation, which the soundness proof must relate through a semantic coupling relation.

The discretized program is shown on the right. The cuts ≤ 0.3, <1.5, ≤ 1.8 induce the intervals $(-\infty, 0.3]$, $(0.3, 1.5)$, $[1.5, 1.8]$, $(1.8, +\infty)$. Each **discrete** statement lists the mass of these intervals. The Gaussian masses are rounded to eight decimal places; the displayed values sum to 1.

2.4 Discrete Latent Variable

Continuous distributions can have parameters that depend on discrete random choices. In this example, a Gaussian distribution's standard deviation is determined by such a random choice:

```
let x = if flip() then 0.5 else 1.5 in gaussian(0, x) < 0.5
```

The type annotations show how the variable x determines the Gaussian's standard deviation, and Figure 19 illustrates how type information is propagated.

(a) Type-Annotated Program

```
let x =
  (if flip() then
      (0.5 : float[{<=0.5,<=1.5}; {0.5}])
    else
      (1.5 : float[{<=0.5,<=1.5}; {1.5}])
  : float[{<=0.5,<=1.5}; {0.5,1.5}])
in
((gaussian(0, x) : float[{<0.5}; T])
  < (0.5 : float[{<0.5}; {0.5}]))
```

(b) Discretized Program

```
let x =
  if flip() then 0 else 1
in
(if x == 0 then
    discrete(0.84, 0.16)
  else
    discrete(0.63, 0.37)) < 1
```

Because x is used as a parameter to the **gaussian** distribution, Slice generates a separate case for each possible value that x can take (namely 0.5 and 1.5). To support this, the type system uses non-strict inequalities in cut sets, allowing these values to map to unique integer indices starting at 0. Specifically, 0.5 maps to index 0 via `float[{<=0.5,<=1.5}; {0.5}]`, and 1.5 maps to index 1 via `float[{<=0.5,<=1.5}; {1.5}]`. The resulting discretized program then branches on these values and instantiates the corresponding discrete distribution, depending on which standard deviation was selected by the initial flip, as shown in the program on the right.

2.5 Conditioning

Slice supports conditioning via **observe**. Consider the following: